Experiment-6

Threads and Interprocess communication

AIM: To understand and implement:

- Thread creation using fork...join
- Differences between join, join_any, and join_none
- Thread control using wait fork and disable fork
- Interprocess communication using:
    - Events
    - Persistent event trigger
    - Mailboxes (bounded and unbounded)
    - Semaphores (resource control)

SOFTWARE/TOOLS USED: Synopsys VCS

QUESTIONS:

PART–A: Threads and Process Control

You are verifying a UART module where three independent tasks execute concurrently.

Tasks:

1. send_data() → delay #4 → prints "Data sent"
2. receive_ack() → delay #7 → prints "Acknowledgment received"
3. log_transaction() → delay #3 → prints "Transaction logged"

Task 1: fork–join Variants

Implement three versions:

1. fork...join
2. fork...join_any
3. fork...join_none

Requirements:

- Print "Transaction started" before fork block.
- Print "Transaction completed" immediately after fork block.
- Record simulation time for each print.
- For join_any, use disable fork after completion.
- For join_none, use wait fork.

Observation Questions

1. At what time does "Transaction completed" print for:
    - join
    - join_any
    - join_none
2. Which construct ensures all tasks complete before proceeding?
3. Why is disable fork required with join_any?
4. When would join_none be preferred in verification?

PART–B: Events and Race Conditions

You are designing a testbench for a memory controller.
Operations:

1. write_data()
    - Takes #5 time units
    - Triggers event write_done
2. read_data()
    - Waits for write_done
    - Prints "Read operation started"
    - Takes #3 time units

Task 2: Standard Event Implementation (instantaneous trigger)

- Declare event write_done
- Trigger the event:
- Use @write_done in read process
- Print simulation timestamps

Task 3: Persistent Trigger Version
Replace trigger with: peristtent trigger and repeat task 2

Analysis Questions

1. What happens if read process begins after event was already triggered?
2. Why does standard event cause race condition?
3. How does persistent trigger solve late-arrival problem?
4. In verification, when are persistent events preferred?

PART–C: Mailbox-Based IPC

Create three classes:

- packet
- generator
- driver

Packet Class

- rand bit [3:0] data
- rand bit [3:0] addr
- display(string msg)
- In post_randomize(), call display("Randomized Packet")

Generator Class

- Contains packet handle
- Contains mailbox handle
- Constructor accepts mailbox
- In start():
    - Create 10 packets
    - Randomize
    - Put into mailbox

Driver Class

- Contains packet handle

- Contains mailbox handle
- Constructor accepts mailbox
- In start():
  - Get 10 packets
  - Display received packets

Task 4: Unbounded Mailbox
- Use mailbox mb = new();
- Execute generator and driver using fork

Analysis Questions
1. What happens if mailbox is full?
2. What happens if mailbox is empty?
3. How does mailbox differ from event?
4. Why is mailbox preferred over event for transaction transfer?

PART–D: Semaphore

You are modelling shared access to a bus.
- Create semaphore with 1 key.
- Create 3 parallel processes attempting to access bus.
- Each process:
  - get()
  - Delay #5
  - put()

Observation Questions
1. What happens if semaphore is initialized with 2 keys?
2. How is semaphore different from mailbox?

THEORY:

1. Fork...Join Constructs

SystemVerilog provides three variants of fork blocks to launch concurrent threads:

Construct	Behaviour	Use Case
fork...join	Parent waits for ALL child threads to complete	Full parallel task completion
fork...join_any	Parent resumes when ANY one thread finishes	Timeout or first-response logic
fork...join_none	Parent continues immediately; all threads run in background	Background monitoring tasks

2. Events

Events in SystemVerilog are synchronisation objects used to signal between concurrent processes. There are two trigger mechanisms:
- Standard Trigger (->): Instantaneous one-cycle pulse. If the receiving process is not waiting at the exact moment, it misses the event — causing a race condition.
- Persistent Trigger (.triggered): Remains active for the rest of the current time step. The receiver can check this at any point within the same time step, avoiding race conditions.

3. Mailboxes
A mailbox is a first-in, first-out (FIFO) queue used to pass data between processes or class objects. It supports two modes:

- Unbounded Mailbox (new()): No size limit; put() never blocks.
- Bounded Mailbox (new(N)): Limited to N entries; put() blocks when full, get() blocks when empty.

Mailboxes transfer complete objects (handles), making them ideal for generator-driver communication in testbenches.

4. Semaphores
A semaphore is a key-based synchronisation object. Processes call get(n) to acquire n keys and put(n) to release them. If insufficient keys are available, get() blocks until they are returned. A semaphore initialised with 1 key implements mutual exclusion (mutex), ensuring only one process accesses a shared resource at a time.

System Verilog Code with Comments (#):

PART A1: Threads and Process Control (fork…join)

```
//-------------------------------------------

// PART A : fork...join

//-------------------------------------------

module fork_join_exp;

    task send_data();

        #4 $display("[%0t] Data sent", $time);

    endtask



    task receive_ack();

        #7 $display("[%0t] Acknowledgment
received", $time);

    endtask



    task log_transaction();

        #3 $display("[%0t] Transaction logged",
$time);

    endtask

    initial begin

        $display("\n========== PART A :
fork...join =========");

        $display("[%0t] Transaction started",
$time);



        fork

            send_data();

            receive_ack();

            log_transaction();

        join



        $display("[%0t] Transaction completed",
$time);

    end

endmodule
```

PART A2: Threads and Process Control (fork…join_any)

```
//-------------------------------------------

// PART A : fork...join_any

//-------------------------------------------

module fork_join_any_exp;


    task send_data();

        #4 $display("[%0t] Data sent", $time);

    endtask


    task receive_ack();

        #7 $display("[%0t] Acknowledgment received", $time);

    endtask


    task log_transaction();

        #3 $display("[%0t] Transaction logged", $time);

    endtask


    initial begin

        $display("\n========== PART A : fork...join_any =========");

        $display("[%0t] Transaction started", $time);


        fork

            send_data();

            receive_ack();

            log_transaction();

        join_any


        $display("[%0t] Transaction completed (first task finished)", $time);


        disable fork;


        $display("[%0t] Remaining tasks terminated using disable fork", $time);

    end


endmodule
```

PART A3: Threads and Process Control (fork…join_none)

```
//-------------------------------------------

// PART A : fork...join_none

//-------------------------------------------

module fork_join_none_exp;
```

```
        task send_data();
                #4 $display("[%0t] Data sent",
$time);
        endtask

        task receive_ack();
                #7 $display("[%0t] Acknowledgment
received", $time);
        endtask

        task log_transaction();
                #3 $display("[%0t] Transaction
logged", $time);
        endtask

        initial begin
                $display("\n========== PART A
: fork...join_none =========");
                $display("[%0t] Transaction started",
$time);

                fork
                        send_data();
                        receive_ack();
                        log_transaction();
                join_none

                $display("[%0t] Transaction
completed (parent continues immediately)",
$time);

                wait fork;

                $display("[%0t] All forked tasks
finished", $time);
        end

endmodule
```

PART–B1: Events and Race Conditions (Standard Event Trigger)

```
//-------------------------------------------
// PART B : Standard Event Trigger
//-------------------------------------------
module event_instantaneous;

        event write_done;

        task write_data();
                $display("[%0t] Write operation
started", $time);
                #5;
                $display("[%0t] Write operation
completed -> triggering event", $time);
```

```
            -> write_done;

        endtask


        task read_data();

            $display("[%0t] Waiting for
write_done event", $time);

            @write_done;

            $display("[%0t] Read operation
started", $time);

            #3;

            $display("[%0t] Read operation
completed", $time);

        endtask

        initial begin

            $display("\n========== PART B
: Instantaneous Event =========");


            fork

                write_data();

                read_data();

            join

        end


endmodule
```

PART–B2: Events and Race Conditions (Persistent Event Trigger)

```
//-------------------------------------------

// PART B : Persistent Event Trigger

//-------------------------------------------

module event_persistent;


        event write_done;


        task write_data();

            $display("[%0t] Write operation
started", $time);

            #5;

            $display("[%0t] Write operation
completed -> triggering event", $time);

            -> write_done;

        endtask


        task read_data();

            $display("[%0t] Waiting using
persistent trigger", $time);

            wait(write_done.triggered);

            $display("[%0t] Read operation
started", $time);

            #3;

            $display("[%0t] Read operation
completed", $time);

        endtask
```

```
    initial begin

        $display("\n========== PART B : Persistent Event =========");



        fork
            write_data();
            read_data();
        join
    end


endmodule
```

PART-B3: Race Condition Demonstration

The following two modules explicitly demonstrate the race condition by delaying the read process past the event trigger point (t=5). The read process begins at t=8, which is after the event has already fired.

```
module event_instantaneous_demo;

    event write_done;

    task write_data();
        $display("[%0t] Write operation started",$time);
        #5;
        $display("[%0t] Write completed -> triggering event",$time);
        -> write_done;
    endtask

    task read_data();
        #8; // read starts AFTER event already happened
        $display("[%0t] Read process started and waiting for event",$time);
        @(write_done);
        $display("[%0t] Read operation started",$time);
    endtask

    initial begin
        $display("\n========== Instantaneous Event Demo =========");
        fork
            write_data();
            read_data();
        join_any

        #20 $display("[%0t] Simulation ending (read never triggered)",$time);
    end
endmodule

module event_persistent_demo;

    event write_done;

    task write_data();
        $display("[%0t] Write operation started",$time);
        #5;
        $display("[%0t] Write completed -> triggering event",$time);
        -> write_done;
    endtask

    task read_data();
        #8;
        $display("[%0t] Read process started",$time);
        wait(write_done.triggered);
        $display("[%0t] Persistent trigger detected -> Read started",$time);
    endtask

    initial begin
        $display("\n========== Persistent Event Demo =========");
        fork
            write_data();
            read_data();
        join
    end

endmodule
```

PART– C: Mailbox-Based IPC

```
//-------------------------------------------
// CLASS : packet
//-------------------------------------------
class packet;

    rand bit [3:0] data;
    rand bit [3:0] addr;

    function void display(string msg);
     $display("[%0t] %s : Addr=%0d Data=%0d", $time, msg, addr, data);
    endfunction

    function void post_randomize();
     display("Randomized Packet");
    endfunction

endclass

//-------------------------------------------
// CLASS : generator
//-------------------------------------------
class generator;

    packet pkt;
    mailbox mb;

    function new(mailbox mb);
     this.mb = mb;
    endfunction

    task start();

     repeat(10) begin

        pkt = new();

        if(pkt.randomize())
            pkt.display("Generator created");

        mb.put(pkt);

        $display("[%0t] Generator -> Packet sent to mailbox", $time);

        #5;

     end
```

```
        endtask

endclass

//-------------------------------------------
// CLASS : driver
//-------------------------------------------
class driver;

        packet pkt;
        mailbox mb;

        function new(mailbox mb);
         this.mb = mb;
        endfunction

        task start();

         repeat(10) begin

                mb.get(pkt);

                pkt.display("Driver received");

         end
        endtask

endclass

//-------------------------------------------
// MODULE : mailbox testbench
//-------------------------------------------
module mailbox_tb;

        mailbox mb;

        generator gen;
        driver drv;

        initial begin
        $display("\n========== PART C : Mailbox IPC ==========");

         mb = new();

         gen = new(mb);
         drv = new(mb);

         fork
                gen.start();
                drv.start();
         join
```

```
        $display("[%0t] Simulation completed", $time);
```

```
        end

endmodule
```

PART– D: Semaphore

```
//-------------------------------------------
// PART D : Semaphore Example
//-------------------------------------------
module semaphore_exp;

   semaphore bus_key;

   //-------------------------------------------
   // Process 1
   //-------------------------------------------
   task process1();
      $display("[%0t] Process 1 requesting bus access", $time);
      bus_key.get(1);
      $display("[%0t] Process 1 acquired bus", $time);
      #5;
      $display("[%0t] Process 1 releasing bus", $time);
      bus_key.put(1);
   endtask

   //-------------------------------------------
   // Process 2
   //-------------------------------------------
   task process2();
      $display("[%0t] Process 2 requesting bus access", $time);
      bus_key.get(1);
      $display("[%0t] Process 2 acquired bus", $time);
      #5;
      $display("[%0t] Process 2 releasing bus", $time);
      bus_key.put(1);
   endtask

   //-------------------------------------------
   // Process 3
   //-------------------------------------------
   task process3();
      $display("[%0t] Process 3 requesting bus access", $time);
      bus_key.get(1);
      $display("[%0t] Process 3 acquired bus", $time);
      #5;
```

```
        $display("[%0t] Process 3 releasing bus", $time);

        bus_key.put(1);

    endtask


    //-------------------------------------------

    // Main

    //-------------------------------------------

    initial begin


        $display("\n========== PART D : Semaphore ==========");


        bus_key = new(1);   // only one process can use bus


        fork

            process1();

            process2();

            process3();

        join


        $display("[%0t] All processes completed", $time);


    end


endmodule
```

RESULTS:

Part A – Fork...Join Variants

A1: fork…join

```
========== PART A : fork...join ==========
[0] Transaction started
[3] Transaction logged
[4] Data sent
[7] Acknowledgment received
[7] Transaction completed
          V C S   S i m u l a t i o n   R e p o r t
```

All three tasks (log_transaction at t=3, send_data at t=4, receive_ack at t=7) run concurrently. The parent waits for all — "Transaction completed" prints at t=7, confirming all tasks finished before the parent resumed.

A2: fork…join_any

```
========== PART A : fork...join_any ==========
[0] Transaction started
[3] Transaction logged
[3] Transaction completed (first task finished)
[3] Remaining tasks terminated using disable fork
          V C S   S i m u l a t i o n   R e p o r t
```

The shortest task (log_transaction, #3) finishes first. The parent resumes at t=3, prints "Transaction completed", then disable fork kills the remaining threads. The output confirms only t=3 events appear.

A3: fork…join_none

```
=========== PART A : fork...join_none ==========
[0] Transaction started
[0] Transaction completed (parent continues immediately)
[3] Transaction logged
[4] Data sent
[7] Acknowledgment received
[7] All forked tasks finished
          V C S   S i m u l a t i o n   R e p o r t
```

The parent continues immediately at t=0 and prints "Transaction completed (parent continues immediately)". All three background tasks complete at their natural times (t=3, t=4, t=7), and wait fork holds the simulation until all finish at t=7.

Part B – Events and Race Conditions

B1: Standard Event (Instantaneous) — Normal Case

```
=========== PART B : Instantaneous Event ==========
[0] Write operation started
[0] Waiting for write_done event
[5] Write operation completed -> triggering event
[5] Read operation started
[8] Read operation completed
          V C S   S i m u l a t i o n   R e p o r t
```

Both processes start at t=0. read_data is already waiting on @write_done when write_data fires at t=5. The event is caught correctly. Read starts at t=5 and completes at t=8.

B2: Persistent Event — Normal Case

```
=========== PART B : Persistent Event ==========
[0] Write operation started
[0] Waiting using persistent trigger
[5] Write operation completed -> triggering event
[5] Read operation started
[8] Read operation completed
          V C S   S i m u l a t i o n   R e p o r t
```

Both processes start at t=0. read_data uses wait(write_done.triggered) and successfully catches the event at t=5. Functionally identical to B1 when no race condition is present, but safe when timing is non-deterministic.

B3: Race Condition Demonstration — Instantaneous Misses the Event

```
=========== Instantaneous Event Demo ==========
[0] Write operation started
[5] Write completed -> triggering event
[8] Read process started and waiting for event
[25] Simulation ending (read never triggered)
          V C S   S i m u l a t i o n   R e p o r t
```

Read process deliberately starts at t=8 — after the event already fired at t=5. With @write_done, the process misses the instantaneous pulse entirely and blocks indefinitely. Simulation ends at t=25 with the read never triggered.

B4: Race Condition Demonstration — Persistent Catches the Late Arrival

```
=========== Persistent Event Demo ==========
[0] Write operation started
[5] Write completed -> triggering event
[8] Read process started
          V C S   S i m u l a t i o n   R e p o r t
```

Same delayed read (starts at t=8), but now using wait(write_done.triggered). The .triggered property was set at t=5 and is still visible at t=8 within the simulation's time step model. Read proceeds immediately at t=8 — the race condition is eliminated.

Part C – Mailbox IPC

```
=========== PART C : Mailbox IPC ==========
[0] Randomized Packet : Addr=9 Data=8
[0] Generator created : Addr=9 Data=8
[0] Generator -> Packet sent to mailbox
[0] Driver received : Addr=9 Data=8
[5] Randomized Packet : Addr=8 Data=15
[5] Generator created : Addr=8 Data=15
[5] Generator -> Packet sent to mailbox
[5] Driver received : Addr=8 Data=15
[10] Randomized Packet : Addr=14 Data=8
[10] Generator created : Addr=14 Data=8
[10] Generator -> Packet sent to mailbox
[10] Driver received : Addr=14 Data=8
[15] Randomized Packet : Addr=10 Data=1
[15] Generator created : Addr=10 Data=1
[15] Generator -> Packet sent to mailbox
[15] Driver received : Addr=10 Data=1
[20] Randomized Packet : Addr=11 Data=12
[20] Generator created : Addr=11 Data=12
[20] Generator -> Packet sent to mailbox
[20] Driver received : Addr=11 Data=12
[25] Randomized Packet : Addr=4 Data=2
[25] Generator created : Addr=4 Data=2
[25] Generator -> Packet sent to mailbox
[25] Driver received : Addr=4 Data=2
[30] Randomized Packet : Addr=9 Data=7
[30] Generator created : Addr=9 Data=7
[30] Generator -> Packet sent to mailbox
[30] Driver received : Addr=9 Data=7
[35] Randomized Packet : Addr=15 Data=9
[35] Generator created : Addr=15 Data=9
[35] Generator -> Packet sent to mailbox
[35] Driver received : Addr=15 Data=9
[40] Randomized Packet : Addr=6 Data=2
[40] Generator created : Addr=6 Data=2
[40] Generator -> Packet sent to mailbox
[40] Driver received : Addr=6 Data=2
[45] Randomized Packet : Addr=15 Data=1
[45] Generator created : Addr=15 Data=1
[45] Generator -> Packet sent to mailbox
[45] Driver received : Addr=15 Data=1
[50] Simulation completed
          V C S   S i m u l a t i o n   R e p o r t
```

The generator creates and randomises 10 packets at intervals of #5, placing each in the unbounded mailbox. The driver concurrently retrieves and displays each packet. The output confirms correct FIFO ordering — every packet sent by the generator at a given timestamp is immediately received by the driver at the same timestamp.

Part D – Semaphore

All three processes request the bus at t=0. Only Process 1 acquires the key immediately. Processes 2 and 3 block. At t=5, Process 1 releases the key — Process 2 acquires it and runs until t=10. Process 3 follows and completes at t=15. Strict sequential, non-overlapping bus access is confirmed.

```
=========== PART D : Semaphore ==========
[0] Process 1 requesting bus access
[0] Process 1 acquired bus
[0] Process 2 requesting bus access
[0] Process 3 requesting bus access
[5] Process 1 releasing bus
[5] Process 2 acquired bus
[10] Process 2 releasing bus
[10] Process 3 acquired bus
[15] Process 3 releasing bus
[15] All processes completed
          V C S   S i m u l a t i o n   R e p o r t
```

OBSERVATION QUESTIONS AND ANSWERS:

Part A – Thread Control

#	Question	Answer
Q1	At what time does "Transaction completed" print for join / join_any / join_none?	join: t=7 (all tasks done). join_any: t=3 (log_transaction finishes first). join_none: t=0 (parent continues immediately; all tasks finish by t=7 via wait fork).
Q2	Which construct ensures all tasks complete before proceeding?	fork...join — the parent blocks until every spawned thread finishes.
Q3	Why is disable fork required with join_any?	join_any resumes the parent when any one thread completes, but the others keep running. disable fork explicitly kills remaining threads to prevent unintended side effects.
Q4	When would join_none be preferred in verification?	For background monitors, scoreboards, or watchdog timers that should run throughout simulation without blocking the main test sequence.

Part B – Events and Race Conditions

#	Question	Answer
Q1	What happens if read process begins after the event was already triggered?	With @write_done (instantaneous), the process misses the event entirely and blocks forever — as proven by the race demo output (Simulation ending, read never triggered). With wait(.triggered), it detects the already-set property and proceeds immediately.
Q2	Why does the standard event cause a race condition?	@write_done only unblocks a process waiting at that exact simulation timestep. Even a 1-timestep delay (e.g., read starts at t=8 vs. trigger at t=5) causes the receiver to miss it permanently.

Q3	How does persistent trigger solve the late-arrival problem?	write_done.triggered stays true for the rest of the current timestep. Any process checking it within that window — or even using it as a level-sensitive wait — can still catch the trigger, as demonstrated in the B4 result.
Q4	In verification, when are persistent events preferred?	Whenever the receiving process scheduling is non-deterministic — e.g., in class-based OOP testbenches, UVM phases, or any environment where the response time of a process cannot be guaranteed relative to the trigger.

Part C – Mailbox

#	Question	Answer
Q1	What happens if the mailbox is full?	For a bounded mailbox (new(N)), put() blocks the sender until the receiver calls get() to free a slot. For the unbounded mailbox used here (new()), put() never blocks.
Q2	What happens if the mailbox is empty?	get() blocks the driver until the generator puts a new item — providing implicit synchronisation without any explicit event or semaphore.
Q3	How does mailbox differ from event?	An event carries no data — it is a pure timing signal. A mailbox carries complete object handles with payload data, maintains ordering (FIFO), and provides blocking on both ends.
Q4	Why is mailbox preferred over event for transaction transfer?	Mailboxes transport full transaction objects, preserve ordering, support buffering, and provide natural flow control — all essential for the generator-driver pattern in structured verification.

Part D – Semaphore

#	Question	Answer
Q1	What happens if the semaphore is initialised with 2 keys?	Two processes can acquire the bus simultaneously. The third is blocked until either of the first two releases a key. Bus access is no longer mutually exclusive.
Q2	How is semaphore different from mailbox?	A semaphore controls access rights (capacity of concurrent access) — no data passes through it. A mailbox transfers

		actual transaction objects between processes. Semaphore = resource guard; Mailbox = message channel.

INFERENCE: This experiment provided comprehensive hands-on understanding of SystemVerilog's concurrent programming and IPC model. Key insights:

- The choice of fork...join variant directly determines the synchronisation point of the parent. fork...join is safest for correctness; fork...join_any and fork...join_none are used when partial or non-blocking concurrency is needed, and require explicit cleanup (disable fork / wait fork).

- ‘The race condition demonstration powerfully illustrated the danger of instantaneous events in non-deterministic environments. The @event syntax should only be used when the receiver is guaranteed to be waiting before the trigger fires. In all other cases, wait(.triggered) is the safer choice.

- Mailboxes provide a clean, object-oriented IPC channel with built-in flow control. The blocking semantics of get() and put() remove the need for manual event-based handshaking between producers and consumers.

- Semaphores correctly enforce mutual exclusion on shared resources. The sequential bus access pattern (Processes 1, 2, 3 each running for exactly #5 with no overlap) confirmed that a 1-key semaphore is equivalent to a hardware mutex.

Together, these constructs form the backbone of layered, class-based verification environments such as UVM, where independent agents must communicate and synchronise reliably without race conditions or deadlocks.